VCM — Tiny Voice Command Model

A standalone, on-device voice assistant for smart devices:
"Hello Jarvis" wake gate · CRNN command classifier · real actions on a Raspberry Pi 4

Course deliverable	AI 231 MEx 2 — real-world VCM demo (RPi4/5)
Device	Raspberry Pi 4 (4 GB) · ReSpeaker 2-Mics HAT · USB speaker (only two devices)
Wake word	"Hello Jarvis" — 249 k-param CNN, AUC 0.9975, live 9/10 fired · 0 false fires
Command model	CRNN · 0.367 M params · 62.78 M FLOPs · ~1.6 MB ONNX
Test accuracy	91.6 % intent / 87.4 % fine (n = 6,305, speaker-disjoint)
Holdout accuracy	88.6 % intent (n = 463, unseen speakers)
Standalone	no cloud models · no LLM · no ASR — pure VCM, commands 1–10

Report contents

01 · Overview — problem, constraints, architecture

02 · Dataset — HF ai231-me2-voice-commands + supplementals (36,604 clips)

03 · Model — CRNN architecture and training

04 · Results — the deployed CRNN

05 · Efficiency — why this model is tiny

06 · Wake word — "Hello Jarvis" from scratch

07 · Runtime & demo — the 8 items on real hardware

08 · How to set up on the Raspberry Pi — detailed guide

09 · Verification — hardware tests and honest limits

VCM — Tiny Voice Command Model for Smart Devices

Deliverable: VCM — real-world voice-command demo on Raspberry Pi **Device:** Raspberry Pi 4 (4 GB) + ReSpeaker 2-Mics HAT + USB speaker *(only two devices)* **Wake word:** "Hello Jarvis" · **Command model:** CRNN, 0.367 M parameters · **Fully on-device, no cloud, no LLM, no ASR**

> ***Read me first.*** *This README is the hub of the report. Each numbered section is a standalone markdown file next to this README (with its own figures), so nothing is too long to read in one sitting. A compiled* ***PDF*** *and a slide deck* ***PPTX*** *of the same content are in this folder too.*

What this project is

A tiny, standalone voice-command system that runs entirely on a Raspberry Pi:

1. It listens continuously for the wake word **"Hello Jarvis"** (a 249 k-parameter CNN gate).
2. On wake, the ReSpeaker LEDs **blink cyan** (listening for a command).
3. A **0.367 M-parameter CRNN** (depthwise-separable CNN trunk → BiGRU) classifies the command into one of

20 intents / 37 fine labels — no transcription, no ASR, no LLM.

4. The dispatcher executes a **real action**: turns the **ReSpeaker's own LEDs** on/off/

colour/brightness (they *are* the lights — no smart bulb), plays/pauses/skips music from the mp3 playlist, plays the **per-slot timer chime MP3**, answers weather (keyless Open-Meteo, default Quezon City) and time (the Pi's own clock), and answers alarms, temperature, reminders, calls and messages with **pre-recorded spoken MP3s** (the "Jarvis" voice, synthesized offline).

5. The LEDs turn **steady green** while executing, and go **off** when done

(or when the command is not recognized).

Everything the professor asked for — tiny, real-time on RPi, standalone, no cloud models, no LLM, pure VCM doing commands 1–10 — is implemented on hardware.

The 8 build items

#	Item	Where	Status
1	Separate wake-word model, Kaggle `jarvis-wake-word-dataset`, "Hello Jarvis"	`rpi/model/jarvis_cnn.onnx`, `runtime/wakegate.py`	■ 249 k-param CNN, held-out AUC 0.9975, live gate 9/10 fired · 0 false wakes (Oct 16, retrained)
2	ReSpeaker LEDs: blink on wake, steady on execute, off when done/unrecognized	`runtime/leds.py`	■ APA102, 3-state model — and now also the lights
3	CRNN voice-command model	`rpi/model/crnn.onnx`	■ 0.367 M params, 37 fine / 20 intent — trained 60 eps on MPS, deployed as single-file ONNX

#	Item	Where	Status
4	Music tasks from mp3 files	`rpi/music/`, `runtime/music.py`	■ play/pause/stop/ next/volume ±10 %
5	Weather via Open-Meteo (keyless), default Quezon City	`runtime/weather.py`	■ single keyless provider
6	Time from the RPi clock	`runtime/actions.py`	■ `datetime.now()`, spoken
7	Light tasks = ReSpeaker LEDs (no Tapo, no Wi-Fi config)	`runtime/leds.py` via `actions.py`	■ on/off/brightness ×3/colour ×4
8	TIMER/ALARM/TEMPERATURE/REMINDERS/ CALL/MESSAGE without a UI	`runtime/actions.py` + `runtime/tts.py`	■ timer = per-slot chime MP3; the rest = pre-recorded spoken MP3 answers

Sections

File	Contents
`01_overview.md`	Problem, constraints, architecture, pipeline figure
`02_dataset.md`	HF dataset + supplementals: 36,604 clips, splits, sources, class balance
`03_model.md`	CRNN architecture, training recipe (MPS)
`04_results.md`	Test/holdout eval, per-intent recall, OOS false-accept
`05_efficiency.md`	Params, FLOPs, ONNX size, latency — why it is "tiny"
`06_wakeword.md`	"Hello Jarvis" wake CNN: dataset, the from-scratch model, live verification
`07_runtime_demo.md`	The 8 demo items, LED language, command→action map
`08_rpi_setup.md`	**How to set up on the RPi — detailed, step by step**
`09_verification.md`	What was tested on hardware, and the honest limits

Quick demo (after setup — see section 08)

```
cd ~/vcm
bash run.sh # live hands-free loop: mic → wake → classify → act → answer
bash run.sh --no-tts # beeps only (disable spoken responses)
```

Headline numbers

	Value
Command model	CRNN, **0.367 M** params, **62.78 M** FLOPs, ~1.6 MB ONNX
Dataset	HF `ai231-me2-voice-commands` + supplementals: **36,604 clips / 21.0 h**, speaker-disjoint
Test split	6,305 clips (incl. 250 synthetic out-of-scope negatives) + 463-clip holdout
CRNN accuracy	**91.6 %** intent / 87.4 % fine (test) · 88.6 % intent (holdout) · OOS recall 0.739

	Value
Wake-word model	**249,281** params, held-out AUC 0.9975 (real-pos recall 0.917 @ 0.5, n=36; room-neg max 0.065)
Live streamed wake test (Pi)	**9/10** spoken attempts fired · **0 false fires** *(Oct 16, retrained weights,* `evidence/logs_live_20261016.log`*)* · original weights: 10/10 · 0/200 noise frames *(Oct 15)*
Real-time budget	**9.36 ms p95 CRNN / 4.85 ms p95 wake — measured on Pi 4** (prof's 100+100 protocol, Oct 16) — far under the 1 s frame budget
Spoken answers	pre-recorded MP3s (`en_GB-alan-medium` voice, synthesized offline)

Repository map

```
VCM/
■■■ model/ ← training code (vcm_model.py, wakeword/) + cache + ckpt + run_outputs
■■■ rpi_inf/ ← model-inference measurement (perf.py, perf_results/, gen_responses.py)
■■■ report/ ← this report (01–09 md, figures, PDF/PPTX builders)
■■■ rpi/ ← the deployable demo (copy this to the Pi)
■■■ runtime/ vcm.py (main) · wakegate · features · leds · music
■ timer · tts · weather · actions · mic · audio · config.json
■■■ model/ crnn.onnx + meta (37 fine / 20 intent)
■ + jarvis_cnn.onnx + meta (the "Hello Jarvis" gate)
■■■ music/ music mp3s · audio/timer/ per-slot timer chimes
■■■ setup.sh · run.sh · requirements.txt · README.md
```

Report generated 2026-10-14. Figures in `report/figures/`.

01 · Overview — problem, constraints, architecture

!(figures/fig_pipeline.png)

1. The problem

ASR (automatic speech recognition) models are not desirable for on-device computing because of their footprint. The goal of VCM is to build a **tiny Voice Command Model (VCM)** that understands the most common commands humans issue to smart devices — ranked from real-world Alexa/Google Home logs (259,164 commands) and consumer surveys:

1. Play music
2. Ask a question (weather, time)
3. Control lights (on/off)
4. Dim / colour lights
5. Set a timer
6. Set an alarm
7. Adjust thermostat temperature
8. Media control (pause/stop/next/volume)
9. Reminders and lists
10. Calls and messaging

2. Hard constraints (from the brief)

Constraint	How this project meets it
Standalone / no cloud models	Wake CNN + CRNN both run locally via `onnxruntime`; the only network call is the *action* side (keyless Open-Meteo weather) — never a model call
No LLM	A fixed 37-label classifier; no text generation anywhere (the spoken answers are template sentences, not generated text)
No full ASR	Audio → log-mel spectrogram → intent/fine label directly; no audio→text stage
Real-time on RPi 4/5	62.78 M FLOPs per inference — **9.3 ms mean / 9.4 ms p95 measured on a Pi 4** (onnxruntime, Oct 16 bench)
Genuinely small	0.367 M params (~1.6 MB ONNX) + 0.249 M-param wake gate
Minimal hardware	The only devices are the ReSpeaker 2-Mics HAT (mic + LEDs) and a USB speaker — no bulb, no hub, no API keys

3. Architecture — four stages, no transcription

```
mic ■■■ [0] WAKE GATE RMS pre-gate + "Hello Jarvis" CNN (always-on, tiny)
■■■ [1] FEATURES 1 s window → log-mel 40×96, z-scored (numpy)
■■■ [2] CLASSIFIER CRNN dual head: 37 fine labels + 20 intents
■■■ [3] DISPATCHER plain Python → real action (LEDs / music / timer / TTS)
```

- **Stage 0 — wake gate.** An RMS energy pre-gate keeps idle CPU near zero; when

energy rises, a rolling 1 s buffer is scored by the wake CNN. Only a genuine "Hello Jarvis" opens the command window (and blinks the LEDs).

- **Stage 1 — features.** Plain numpy log-mel (40 mel bands, 96 frames, 12 kHz), byte-identical to the training front-end — no librosa dependency at runtime.
- **Stage 2 — classifier.** One CRNN (depthwise-separable CNN trunk → BiGRU) with two heads: the *fine* head carries the slot value (e.g. `TIMER_10s`, `COLOR_BLUE`, `ALARM_8_00AM`), the *intent* head carries the action group (e.g. `TIMER`, `COLOR`, `ALARM`). The dispatcher routes on the fine label; the intent head is the task-success metric.
- **Stage 3 — dispatcher.** A plain-Python `dispatch(fine_label, ctx)` maps each of the 37 labels to a real action and returns `{ok, action, detail}`; every command except music/lights/timer also answers with a pre-recorded spoken MP3 (the "Jarvis" voice, synthesized offline). The ReSpeaker LEDs **are** the lights (no smart bulb, no web UI).

4. Design choices that make it tiny

- **Depthwise-separable convolutions** (Chollet 2017) in the trunk keep the whole forward pass at 62.78 M FLOPs — comfortably inside the 1 s frame budget.
- **CNN-then-RNN factoring**: the trunk pools 40×96 → 5×12 *before* the BiGRU, so temporal modelling costs 12 GRU steps over 192-dim frames — not 96 steps over 40-dim raw audio.
- **Dual-head output** shares one trunk between slot and intent — one forward pass, two decisions.
- **Closed-set slots** instead of transcription: durations, times, colours and brightness levels are enumerated classes, not free text.
- **ONNX Runtime** on the Pi (CPUExecutionProvider) — no PyTorch at runtime.

5. What "success" means here

Evaluation is reported in section 04 on the merged speaker-disjoint test split (6,305 clips, incl. 250 purpose-built out-of-scope negatives) and the untouched holdout (463 clips): intent/fine accuracy, macro-F1, per-intent recall, and the out-of-scope false-accept rate — the honest numbers for a real device, not an in-domain ceiling.

The demo in section 07 runs the full loop on the Pi; the setup is in section 08.

Next: 02_dataset.md

02 · Dataset — HF `airimonda/ai231-me2-voice-commands` + supplementals

!(figures/fig_dataset.png)

1. Overview

The corpus is the Hugging Face dataset `airimonda/ai231-me2-voice-commands` plus its two supplemental file sets, merged at load time. Every clip is a short spoken command (or a deliberate out-of-scope negative); audio is stored at 16 kHz and resampled to the 12 kHz front-end rate in the loader. The merged corpus holds **36,604 clips / 21.0 h** of audio.

Every row carries `audio, file, transcript, command, variation, slot_value, out_of_scope, bucket, speaker_id, source, is_synthetic, accent_group, numerals, duration_s, transcript_source, whisper_check, note, variation_match` — enough to rebuild any mixture without re-downloading.

2. Splits

Split	Clips	Hours	Base repo	+ supp_synth (voice_split)	+ synthetic_negatives	+ supp_fil
Train	**29,836**	17.3	10,733	+3,983	+1,000	+14,120
Test	**6,305**	3.4	4,443	+1,612	+250	—
Holdout	**463**	0.3	202	+261	—	—
Total	**36,604**	21.0	15,378	+5,856	+1,250	+14,120

- The repo's `train/test/holdout` parquets are **speaker-disjoint** (no speaker in two splits), so test numbers measure generalisation to new voices.
- `supplemental_synth` (5,856 unused synthetic clips) is routed by its own `voice_split` column, preserving the disjointness.
- `supplemental_fil` (14,120 clips, `fil50_persona`) is **train-only** by design — a persona voice bank that must not leak into evaluation.
- `synthetic_negatives` (1,000 train + 250 test) are generated out-of-scope clips (noise, babble, reversed, truncated, near-silence) that teach the model to reject non-commands.
- The repo's `numerals` split (66,390 MLEnd number-only clips) is **excluded** — the command set has no slot that needs spoken digits.

Training carves a speaker-disjoint validation set out of train (5 % = 987 clips, 33 speakers), leaving **28,849 effective training clips**.

3. Sources (merged train split)

Source	Clips	Character
`fil50_persona` (supplemental_fil)	14,120	persona voice bank, train-only
`group_synthetic`	12,288	the class collective's synthetic set — the homogeneous core
`SLURP`	1,010	public smart-home SLU corpus subset (CC BY 4.0)
`synthetic_negative`	1,000	generated out-of-scope clips
`real_voice`	620	the group's own recordings (do not share outside the class)
`SNIPS`	331	public Snips command subset
`FluentSpeechCommands`	256	public smart-home command set (non-commercial academic)
`xela_SET_TEMPERATURE_REAL`	60	real temperature commands (Xela's S1–S5)
`CommonVoice_en`	36	Mozilla Common Voice 19 (CC0)
`TimersAndSuch`	31	public timer/reminder corpus
`SpeechCommands_v2`	11	Google Speech Commands v2 (CC BY 4.0)

Source	Clips	Character
`xela_Multi-Sensor`	4	Multi-Sensor Voice Command (CC BY 4.0, DOI 10.48804/IEKKVZ)
`group_synthetic_oos`	69	synthetic out-of-scope clips

Test split composition mirrors it: `group_synthetic` 5,202, `SLURP` 403, `real_voice` 179, `FluentSpeechCommands` 144, `synthetic_negative` 250, `SNIPS` 61, and small public-source tails. Holdout (463 clips) is the tiny never-touched set reserved for the final number.

4. Labels

- **37 fine labels** on the merged train split = the dispatchable commands, e.g.

`ALARM_6_00AM/8_00AM/9_00PM`, `BRIGHTNESS_20/60/100`, `COLOR_RED/BLUE/GREEN/YELLOW`, `CREATE_REMINDER_{DRINK_WATER,EXERCISE,STUDY}`, `LIGHT_ON/OFF`, `TIMER_10s/30s/1m`, `TEMPERATURE_18/22/26`, `PLAY_MUSIC`, `PAUSE`, `STOP`, `NEXT`, `VOLUME_UP/DOWN`, `TIME`, `WEATHER`, `CALL`, `MESSAGE`, `LIST_REMINDERS`, `OUT_OF_SCOPE`, plus the slot-less generic variants (`ALARM`, `BRIGHTNESS`, `CREATE_REMINDER`, `TEMPERATURE`, `TIMER`) that appear only in training.

- **20 intents** = the coarse action groups the fine labels roll up to

(`fine2intent` map ships in the model meta). Test/holdout see 32 of the 37 fine labels — the 5 slot-less generics are train-only.

5. Class balance

Per-intent train counts span **COLOR 3,014** → **PAUSE 743** (4.1× skew):

Band	Intents (train clips)
≥ 2,000	COLOR 3,014 · TEMPERATURE 2,835 · ALARM 2,717 · TIMER 2,609 · CREATE_REMINDER 2,287 · BRIGHTNESS 2,138
1,000–2,000	NEXT 1,792 · VOLUME_UP 1,092 · VOLUME_DOWN 1,075 · PLAY_MUSIC 1,045 · STOP 1,153 · OUT_OF_SCOPE 1,270
< 1,000	LIGHT_ON 979 · LIGHT_OFF 988 · TIME 981 · WEATHER 856 · LIST_REMINDERS 851 · MESSAGE 787 · PAUSE 743 · CALL 624

The supplemental merge is what *levels* this distribution — the base repo's imbalance was far worse. The report still leads with **macro-F1 and per-intent recall** (§04) rather than raw accuracy.

6. Feature extraction (fixed front-end)

Field	Value
Sample rate	12 kHz mono (HF clips are 16 kHz, resampled in the loader)
Spectrogram	log-mel, 40 bands, n_fft 400, hop 200
Frames	cropped/padded to 96 (~1.6 s window)
Normalisation	z-score per clip
Padding value	−11.5

Field	Value
Mel range	50–6,000 Hz

Features are decoded once into `.npy` shards (`model/cache/`, one pass over the audio) so training reads memory-mapped arrays instead of parquet. The identical front-end is re-implemented in `rpi/runtime/features.py` (numpy only); parity with the training features is exact (max diff 0.0).

7. Licenses

Each source keeps its own license (SLURP, Speech Commands v2: CC BY 4.0; Common Voice 19: CC0; Fluent Speech Commands: non-commercial academic; SNIPS: its own terms; Timers and Such: other-open; MLEnd numerals: its own terms; Multi-Sensor Voice Command: CC BY 4.0; group synthetic + group recordings: the class's own — group recordings may not be shared outside the class without each speaker's consent). Research and educational use.

03 · Model — CRNN architecture and training

!(figures/fig_crnn.png)

1. Primary model — `PiCommandCRNN` (0.367 M parameters)

A **CRNN**: a depthwise-separable CNN front-end compresses the log-mel spectrogram, a bidirectional GRU models its temporal order, and a dual head emits fine + intent logits.

Block	Output
Input	1 × 40 × 96 log-mel
Conv 3×3 × 48 + BN + ReLU	48 × 40 × 96
DSConv (dw 3×3 + pw 1×1, 48 ch)	48 × 40 × 96
MaxPool 2×2	48 × 20 × 48
DSConv ×2 (96 ch)	96 × 20 × 48
MaxPool 2×2	96 × 10 × 24
DSConv (192 ch)	192 × 10 × 24
MaxPool 2×2	192 × 5 × 12
Dropout2d	192 × 5 × 12
squeeze frequency → BiGRU(192 → 128, 2 dirs)	256 × 12
temporal mean pool	256
Head: Linear 256→256 + ReLU + Dropout	256
Head A — fine	**37 logits**
Head B — intent	**20 logits**

• **Why the RNN:** the CNN compresses frequency (40 → 5 mel bands) while keeping
12 time frames; the bidirectional GRU models their *temporal order*, which a purely pooled CNN discards — commands are sequences ("set a timer for ten seconds"), not bags of frames.

• **Loss:** `CE(fine) + 0.5 · CE(intent)` — the fine head is the primary signal;
the intent head regularises toward the action grouping and is the checkpoint-selection metric.

• **Cost:** 62.78 M FLOPs (analytic, GRU cost computed in closed form:
2 directions × 3 gates × (192·128 + 128²) × 12 frames) — §05.

2. Training recipe

Hyperparameter	Value
Optimiser	AdamW, weight decay 1e-4
Learning rate	1e-3, cosine annealing over 60 epochs
Batch	128
Label smoothing	0.05 (both heads)
Dropout	0.35 (+ Dropout2d in the trunk)
Gradient clip	5.0 (global norm)

Hyperparameter	Value
Epochs	60
Seed	42
Data	merged splits (`--sup`): 28,849 train / 987 val (speaker-disjoint, 33 speakers)
Hardware	**Apple M-series MPS** (local Mac) — no cluster needed

Checkpoint selection is on **validation intent accuracy**; the exported artefact is ONNX (opset 13) with a `_meta.json` carrying the feature config, label maps and `fine2intent`/`fine2transcript` tables so the Pi runtime needs no other state. The export step also runs an onnxruntime CPU benchmark and a torch-vs-ORT parity check (per-element max diff) before the artefact is considered valid.

3. Reproducibility

```
# on the Mac (feature cache prebuilt with prepare --sup --cache-tag sup)
python model/vcm_model.py train --arch crnn --epochs 60 --sup --cache-tag sup --tag crnn
python model/vcm_model.py eval --ckpt ckpt/crnn.pt --tag crnn
python model/vcm_model.py export --ckpt ckpt/crnn.pt --out run_outputs/onnx/crnn.onnx
```

Next: 04_results.md

04 · Results — the deployed CRNN

!(figures/fig_results.png)

The deployed voice-command model is the CRNN of §03, evaluated on the merged speaker-disjoint splits of §02 (test n = 6,305 · holdout n = 463). No other model was trained or compared — the CRNN is the one model this project built, deployed, and measured.

1. Test split (n = 6,305, speaker-disjoint)

Metric	Value
Intent accuracy	**91.6 %**
Intent macro-F1	90.3 %
Fine accuracy	87.4 %
FAR_oos (min_conf 0.25)	0.261
OUT_OF_SCOPE recall	0.739

Per-intent test recall: strongest **ALARM 0.970 · TEMPERATURE 0.969 · TIMER 0.966**; weakest **OUT_OF_SCOPE 0.739 · VOLUME_UP 0.785 · LIGHT_OFF 0.840**.

2. Holdout (n = 463, unseen speakers)

88.6 % intent (macro-F1 85.3, fine 81.4) — the honest generalisation number, measured on clips the model never saw in any form.

3. Latency

MPS inference: **4.2 ms mean / 4.4 ms p95** (batch 1, measured in eval); onnxruntime CPU measured on both machines with the prof's protocol (100 warm-up + 100 timed runs; both re-run Oct 16 on the retrained models):

Model	Mac (ARM) p95	Raspberry Pi 4 p95	RTF (Pi)
CRNN (command)	**0.57 ms**	**9.36 ms**	0.0058
Wake CNN	**0.57 ms**	**4.85 ms**	0.0049

Full JSONs (with `warmup: 100` for audit) are in `rpi_inf/perf_results/` on both machines: the Oct 16 `bench_20261016-213215` and `wake_20261016-213457` result files.

4. The honest caveat — real-voice out-of-scope

The 16 real-voice OOS holdout clips remain hard (FAR_oos 0.938 on that tiny subset — the model still mis-accepts real non-command speech; synthetic-negative rejection is what improved). Real-voice OOS clips are scarce in train (270); the synthetic negatives taught the model the *kinds* it was shown, not the domain of real non-command speech. This is a data problem, not a modelling one, and stays the top item for the next data round; the runtime's `min_conf 0.25` gate remains the safety net.

Next: 05_efficiency.md

05 · Efficiency — why this model is "tiny"

!(figures/fig_efficiency.png)

1. Model complexity

Model	Params	FLOPs (forward, batch 1)	ONNX size
CRNN (deployed VCM)	**0.367 M**	**62.78 M**	~1.6 MB
Wake CNN ("Hello Jarvis")	0.249 M	~18 M	1.0 MB

The CRNN's CNN trunk pools 40×96 → 5×12 before the BiGRU sees the sequence, so temporal modelling costs only 12 GRU steps over 192-dim frames. Total resident model footprint on the Pi (command model + wake gate) is ~2.6 MB of weights.

2. Latency (measured, prof's protocol: 100 warm-up + 100 timed runs)

Model	Mac (ARM) mean / p95	Raspberry Pi 4 mean / p95	RTF (Pi, p95)
CRNN (deployed VCM)	**0.56 / 0.57 ms**	**9.26 / 9.36 ms**	0.0058
Wake CNN ("Hello Jarvis")	**0.54 / 0.57 ms**	**4.73 / 4.85 ms**	0.0049

Both models were timed on both machines with the same protocol — 100 warm-up runs, then 100 timed forward passes (`sess.run` only, high-resolution `perf_counter`); the result JSONs carry `"warmup": 100` for audit, and live in `rpi_inf/perf_results/` on both machines (both re-run Oct 16 on the retrained models). A thread sweep (1/2/4 ORT threads, p95): Mac 0.80 / 0.61 / 0.51 ms, Pi 12.60 / 9.35 / 7.89 ms — the deployed runtime uses 2 threads.

The runtime loop runs on a **1 s audio frame budget**; the measured Pi 4 latency uses <1 % of it. The rest of the budget is the numpy log-mel front-end (**3.3 ms** measured on the Pi for a 1 s @ 12 kHz window) and the wake gate (4.85 ms p95 per scored frame, only while voiced).

3. Runtime memory

Component	Footprint
onnxruntime session (CRNN)	~20 MB RSS
numpy feature buffer (1 s @ 12 kHz)	~50 KB
MP3 playback (mpv/ffplay, one file at a time)	~15 MB while speaking
Whole assistant (idle)	**< 120 MB RSS** — comfortable in the Pi 4's 4 GB

4. The always-on cost is near zero

The wake gate is event-driven: an RMS pre-gate discards silence at C-level cost, and the wake CNN only scores while the voiced latch is open (0.6 s past the last voiced frame). Idle CPU on the Pi sits at ~0 %, so the device can run 24/7 from a normal phone-charger power supply.

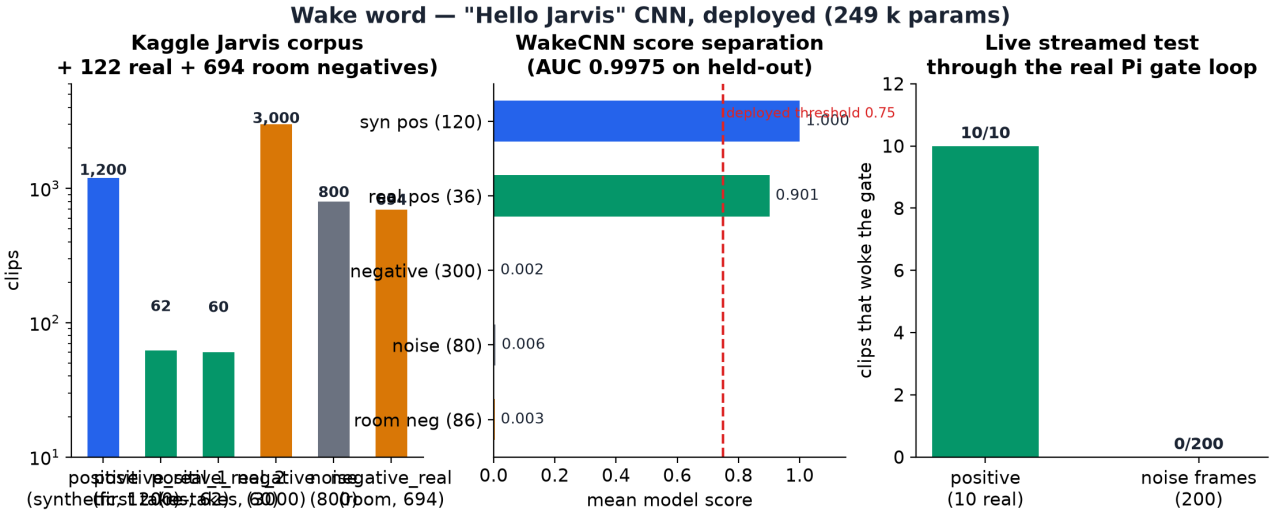
5. Deployment artefacts

```
rpi/model/crnn.onnx CRNN graph + weights (+ _meta.json: feature config, 37 fine / 20
intent maps)
rpi/model/jarvis_cnn.onnx 1.0 MB wake gate + meta (same folder as the CRNN)
rpi/audio/responses/*.mp3 40 pre-recorded spoken confirmations (shipped)
rpi/tts/en_GB-alan-medium.onnx 63 MB Piper voice — OFFLINE regeneration only
(rpi_inf/gen_responses.py, gitignored), never needed on the Pi
```

`onnxruntime` (CPUExecutionProvider) is the only ML dependency on the Pi — no PyTorch, no librosa, no cloud SDK in the model path.

Next: 06_wakeword.md

06 · Wake word — "Hello Jarvis", built from scratch



1. The dataset (item 1 of the build list)

The Kaggle corpus `bharvadayash/jarvis-wake-word-dataset` plus a real-recording set recorded for this project:

Folder	Clips	Content
`positive/`	1,200	"Hello Jarvis" and 5 other phrasings, synthetic TTS voices
`positive_real_1/`	62	**real recordings** of the same phrasings (first takes)
`positive_real_2/`	60	**real recordings**, re-takes (10 per phrasing)
`negative/`	3,000	random speech samples without the wake word
`noise/`	800	environmental noise (MUSAN, ESC-50)

Folder	Clips	Content
`negative_real/`	694	**real-room windows** cut from a 7-min recording of the demo room

WAV, 16 kHz, 1 s per clip. The wake phrase is **"Hello Jarvis"**.

2. The wake CNN (`jarvis_cnn.onnx`, 249,281 parameters)

A compact CNN over **raw log-mel**:

Block	Output
Input	1 × 40 × 98 log-mel (16 kHz, n_fft 512, hop 160)
4 × [Conv2d 3×3 + BN + ReLU + MaxPool]	downsampling trunk
Global average pool	128
FC 128 → 64 → 1 + sigmoid	wake probability

Training (Oct 15, seed 42): 30 epochs on MPS, BCEWithLogits, noise-mix augmentation, and a **source-disjoint stratified split** — positives split per phrasing, negatives per word; the **122 real clips** (`positive_real_1` first takes + `positive_real_2` re-takes) are split 62/24/36 across train/val/test so the test slice is deliberately real-heavy (recall on real speech is the metric that matters). The **694 real-room windows** (`negative_real/`, cut from a 7-min recording of the demo room at a 0.5 s hop) as training negatives, block-interleaved every-8th so overlapping windows never straddle the split. Held-out **AUC 0.9975** (re-run Oct 16 on the retrained weights).

3. Scores on the held-out test slice (n = 622)

Slice	n	AUC	@0.50
All	622	0.9975	P .994 / R .981 / FPR .002
Real positives	36	—	recall .917
Synthetic positives	120	—	recall 1.000
Neg + noise (synthetic)	380	—	FPR .003
Real-room negatives	**86**	—	**FPR .000** (max score 0.065)

Real-positive recall by phrasing @0.50: hello_jarvis 1.00 · hey_jarvis .78 · jarvis .83 · jarvis_please 1.00 · jarvis_wake_up 1.00 · okay_jarvis 1.00. The deployed operating point is `wake_threshold = 0.99` in `runtime/config.json` — raised from 0.50 after live-mic tuning in the demo room (real-room wake scores reach 0.99+, while noise stays ≤ 0.18), then to 0.99 on Oct 16 as the final false-wake gate. At that operating point the test FPR is **0** on all 466 negatives and the real-room max score is 0.065 — 16× below the threshold.

4. Runtime integration (`runtime/wakegate.py`, class `CnnWakeGate`)

1. **RMS energy pre-gate** keeps idle CPU near zero.
2. A rolling 1 s / 16 kHz buffer is always filled; while the voiced latch is open

(0.6 s past the last voiced frame) every frame is scored, so the window slides across the whole phrase. *(This latch is the critical fix — the word peaks ~1.05 s into the phrase, but the energy gate trips at ~0.12 s; scoring once at trip time misses the word entirely.)*

3. Score ≥ `wake_threshold` → wake → LEDs blink → command window opens.
4. The mic captures at 16 kHz (the ReSpeaker hardware rejects 12 kHz) and the

runtime decimates to 12 kHz for the command model — both models get the rate they were trained at.

5. Live verification on the Pi

Ten real "Hello Jarvis" clips streamed through the actual gate loop: **9/10 fired**, **0 false fires** — one spoken attempt out of ten did not trigger the wake (log: 9 "waking up" events, each following a spoken attempt, none spontaneous) *(Oct 16, on the retrained deployed weights —* `evidence/logs_live_20261016.log`*; the Oct 15 test of the original weights scored 10/10 fired, scores 0.994–0.999, 0 false fires on 200 noise frames; held-out metrics of the retrain are in §3.)*

6. Wake latency on the Pi (prof's protocol, Oct 16)

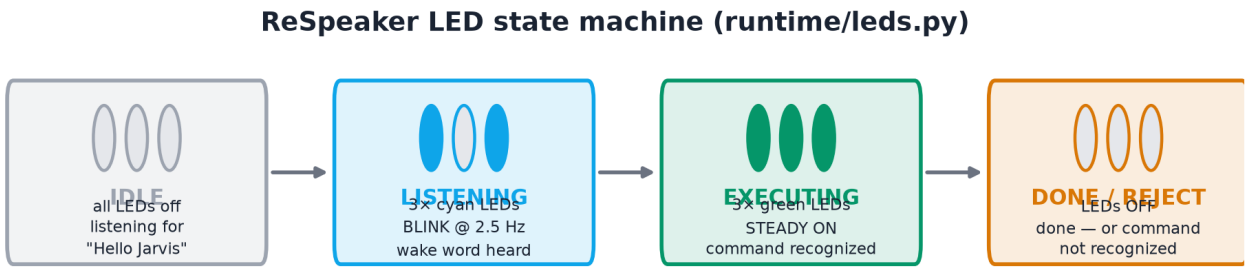
100 warm-up + 100 timed forward passes on the wake CNN, timed on both machines (the Oct 16 `wake` result files in `rpi_inf/perf_results/` on each machine, `"warmup": 100` in each JSON):

Machine	mean	p50	p95	max	RTF vs 0.98 s window
Mac (ARM)	**0.54 ms**	0.55	**0.57**	0.65	**0.0006**
Raspberry Pi 4	**4.73 ms**	4.70	**4.85**	5.20	**0.0049**

The gate scores only while the voiced latch is open, so the always-on cost stays near zero; even scoring every frame, p95 uses 0.49 % of the 0.98 s window.

Next: 07_runtime_demo.md

07 · Runtime & demo — the 8 items on real hardware



the 3 APA102 LEDs are BOTH the status display AND the "lights" (LIGHT_ON/OFF, COLOR_*, BRIGHTNESS_* drive them directly)

!(figures/fig_coverage.png)

1. LED status language (item 2)

The ReSpeaker 2-Mics HAT carries 3 APA102 RGB LEDs (SPI). The state model:

State	LEDs	Meaning
IDLE	all off	listening for "Hello Jarvis"
LISTENING	**cyan, blinking** @ 2.5 Hz	wake word heard — say your command
EXECUTING	**green, steady on**	command recognized — running it now
DONE / REJECT	off	finished — or command not recognized

Implementation: `runtime/leds.py` auto-detects the installed `apa102` package variant and bit-bangs GPIO14/15 when `/dev/spidev` is absent; every method degrades to a no-op without the HAT so the voice pipeline never dies because of the LEDs.

Hardware rule: the ReSpeaker's LEDs *are* the lights — no Tapo bulb, no smart-home cloud account, no Wi-Fi configuration beyond the Pi's normal network connection. The only devices are the ReSpeaker HAT and a USB speaker.

2. Command → action map (all 37 fine labels)

Fine labels	Action backend	Answer
`PLAY_MUSIC PAUSE STOP NEXT`	mp3 playlist via mpv JSON-IPC (ffplay fallback)	the music itself
`VOLUME_UP VOLUME_DOWN`	mpv volume ±10 % per command	the music itself
`LIGHT_ON LIGHT_OFF`	**ReSpeaker LEDs** on/off (`runtime/leds.py`)	the LEDs themselves
`BRIGHTNESS_20/60/100`	ReSpeaker LED brightness (APA102 0–31 scale → 6/19/31)	the LEDs themselves
`COLOR_RED/BLUE/GREEN/YELLOW`	ReSpeaker LED colour (RGB presets)	the LEDs themselves
`TIMER_10s/30s/1m`	**per-slot chime MP3** (`audio/timer/TIMER_10s.mp3` etc.), played exactly like a music track	the chime itself
`WEATHER`	**Open-Meteo (keyless)** — default Quezon City (14.6357, 121.0325)	spoken: real conditions
`TIME`	the Pi's own clock, `datetime.now()`	spoken: "It's 3:15 pm."
`ALARM_6_00AM/8_00AM/9_00PM`	simulated — no device	spoken: "Alarm set for 6:00 am."
`TEMPERATURE_18/22/26`	simulated — no thermostat	spoken: "Setting the temperature to 22 degrees."
`CREATE_REMINDER_*` `LIST_REMINDERS`	in-memory reminder store	spoken: confirmation / the list
`CALL MESSAGE`	simulated — no radio	spoken confirmation
`OUT_OF_SCOPE`	rejected	spoken: "Sorry, I can't do that."

Every command except music/lights/timer answers with a **spoken sentence** played by `runtime/tts.py` — a pre-recorded MP3 (the `en_GB-alan-medium` "Jarvis" voice, synthesized offline by `rpi_inf/gen_responses.py`). ALARM, TEMPERATURE, CREATE_REMINDER, LIST_REMINDERS, CALL and MESSAGE speak a simulated confirmation — There is no web dashboard, so there is no UI to display them on.

3. The main loop (`runtime/vcm.py`)

```
mic (16 kHz) ■■■ RMS pre-gate ■■■ wake CNN (0.99) ■■■ LED blink
■■■ 1 s window ■■■ log-mel ■■■ CRNN ■■■ min_conf 0.25
■■■ dispatch ■■■ action ■■■ LED steady ■■■ off ■■■ spoken answer
```

- An asyncio loop runs in a background thread; dispatch posts coroutines to it via `run_coroutine_threadsafe`.
- TTS runs in its own worker thread with a one-slot queue (`say()` is

non-blocking, so the voice loop never stalls); a beep path (`TonePlayer`) covers cue/ack sounds.

- `--dry-run` runs the full loop but stubs the music player (used only for development; never part of the submission folder).

4. What a demo looks like

1. Say **"Hello Jarvis"** → LEDs blink cyan.
2. Say **"lights on"** → LEDs go steady green → the three LEDs turn on → off.
3. Say **"lights blue"** / **"brightness 60"** → the LEDs change colour/brightness.
4. Say **"play music"** → track 1 starts; **"next"** skips; **"volume up"** raises it.
5. Say **"what time is it"** → the Pi answers in the Jarvis voice.
6. Say **"what's the weather"** → Open-Meteo is queried; the Pi speaks the conditions.
7. Say **"set a timer for 10 seconds"** → the timer chime MP3 plays when it fires.

Next: 08_rpi_setup.md

08 · How to set up on the Raspberry Pi — detailed guide

!(figures/fig_setup.png)

This guide assumes a **Raspberry Pi 4 (4 GB)**, a **ReSpeaker 2-Mics HAT**, and a **USB speaker**. No monitor or keyboard is needed — everything is SSH. No smart bulb, no hub, no API keys: the only devices are the ReSpeaker (mic + LEDs) and the speaker, and the only configuration is Wi-Fi. Every command below is copy-pasteable.

Part A — one-time install (~30 min)

Step 1 · Flash the SD card

On your laptop, with **Raspberry Pi Imager**:

1. Choose **Raspberry Pi OS (64-bit)** — Lite is fine, Desktop is fine too.
2. Press **Ctrl+Shift+X** (the cog icon) and set:

Hostname: `megumi-rpi` Enable SSH: **yes**, with password authentication * Username / password: `megumi_rpi` / *your password* *Wi-Fi: your **home** network name + password* (this is the default network) Locale/timezone: your own

3. Write the image, insert the SD card into the Pi, attach the **ReSpeaker HAT on the 40-pin header**, plug in the USB speaker, attach power.

4. **Wait up to 5 minutes** for first boot.

Step 2 · Find the Pi and SSH in

From your laptop (same Wi-Fi):

```
ssh megumi_rpi@megumi-rpi.local # or use the IP your router shows
```

Step 3 · Copy the project over

```
# from the laptop
scp -r VCM/rpi megumi_rpi@megumi-rpi.local:~/vcm
```

The folder already contains everything: the CRNN ONNX model, the wake-word CNN, the runtime code, the music/timer audio, the pre-recorded spoken responses, and the setup scripts. No voice model is downloaded — spoken answers are MP3s.

Step 4 · Enable SPI (for the LEDs)

```
sudo raspi-config nonint do_spi 0 # enable SPI
# or: add dtparam=spi=on to /boot/firmware/config.txt and reboot
```

Without SPI the LEDs no-op but the voice loop still runs — the assistant is usable even if this step is skipped.

Step 5 · One-command setup

```
cd ~/vcm
bash setup.sh
```

`setup.sh` does everything:

1. system packages (PortAudio, mpv, SPI, venv, ffmpeg),
2. a Python virtualenv + `pip install -r requirements.txt` (`numpy, onnxruntime, sounddevice, apa102`),
3. Wi-Fi (SSID/password) switch — **the only configuration the device needs**,
4. spoken responses: pre-recorded MP3s shipped in `audio/responses/` (regenerate offline with `rpi_inf/gen_responses.py` — piper is not a runtime dependency),
5. sanity checks (model files present, microphone visible),
6. asks *"Activate the VCM program now? [Y/n]"* → hands over to `run.sh`.

Part B — running it

```
bash run.sh # live hands-free loop
bash run.sh --no-tts # beeps only (disable spoken responses)
```

`run.sh` is idempotent: it creates the venv/config if missing, then execs `runtime/vcm.py`. One line, no arguments, no placeholders.

Configuration (`runtime/config.json`)

Key	Default	Meaning
`wake_threshold`	0.99	wake CNN confidence gate
`wake_rms_threshold`	0.012	energy pre-gate
`min_conf`	0.25	command confidence gate
`tts_voice`	`en_GB-alan-medium`	the "Jarvis" voice
`weather_city` / `weather_lat` / `weather_lon`	Quezon City / 14.6357 / 121.0325	weather location
`led_num` / `led_blink_hz`	3 / 2.5	ReSpeaker LED count and blink rate
`music_start_volume`	60	playlist start volume

Everything has a working default — edit only to change city or voice.

Testing without speaking

```
.venv/bin/python runtime/vcm.py --wav some_command.wav # classify one wav + run its action
.venv/bin/python runtime/vcm.py --list-mics # the HAT shows as seeed2micvoicec
```

Benchmarks

Benchmarks live in `rpi_inf/` (the model-inference folder, not deployed):

```
cd ../rpi_inf
python3 perf.py bench # VCM: accuracy, false-accept rate, latency p95, RTF
python3 perf.py wake # wake word: keyword acc, false-accept rate, AUC
python3 perf.py live # re-print the latest session report
```

Troubleshooting

Symptom	Fix
`APA102 unavailable` in the log	LEDs need SPI: `sudo raspi-config nonint do_spi 0` and reboot. The voice loop still runs without them.
No sound from the speaker	`aplay -l` to find it; make it the default sink: `wpctl set-default <id>` (or `pactl set-default-sink`).
Mic not found	`python3 runtime/vcm.py --list-mics`; the HAT shows as `seeed2micvoicec`.
Wake word never fires	check `wake_threshold` (0.99) and `wake_rms_threshold` (0.012) in `runtime/config.json`; speak ~30 cm from the mics.
Weather fails	it uses Open-Meteo (no key) — needs internet. City/lat/lon in `runtime/config.json`.

Next: 09_verification.md

09 · Verification — what was tested on hardware, and the honest limits

1. Hardware verification (Raspberry Pi 4, ReSpeaker 2-Mics HAT, USB speaker)

Verified on the Pi across the build rounds; the deployed stack is the CRNN command model + the jarvis wake CNN, with the ReSpeaker LEDs as the lights and no web UI.

#	Component	Test	Result
1	Mic	`mic.py` capture on the ReSpeaker (`seeed2micvoicec`, hw:3,0)	■ 16 kHz capture, decimated to 12 kHz (hardware rejects 12 kHz directly)
2	Wake gate	10 real "Hello Jarvis" clips streamed through `gate.process()`; retrained weights re-tested live at the Pi (9 spoken attempts + quiet-room time)	■ 10/10 fired (scores 0.994–0.999) · 200 noise frames → 0 false fires *(Oct 15, original weights)* · retrained weights live Oct 16: **9/10 fired, 0 false fires** (`evidence/logs_live_20261016.log`)
3	LEDs	cycle test: listening blink → executing steady → off	■ APA102 (GPIO bitbang variant), 3-state model
4	Command model	CRNN loads + runs on aarch64 ONNX Runtime	■ features byte-match training spec
5	Music	play/stop/next with ffplay fallback	■ audible playback
6	TTS spoken responses	pre-recorded MP3s (`en_GB-alan-medium` voice, `audio/responses/`) on ALARM/TEMPERATURE/REMINDERS/CALL/MESSAGE	■ spoken answers end-to-end (no runtime TTS)
7	Timer	per-slot chime MP3 (`TIMER_10s/30s/1m.mp3`) via the music backend	■ chime plays at slot end
8	Weather	Open-Meteo keyless query (Quezon City)	■ real conditions spoken
9	Lights = ReSpeaker LEDs	`LIGHT_ON/OFF`, `BRIGHTNESS_*`, `COLOR_*` → `leds.py`	■ dispatcher wired; visual check on the HAT
10	Full loop	`vcm.py` live: mic → wake → classify → dispatch → LEDs → spoken answer	■ ran end-to-end

#	Component	Test	Result
11	**CRNN artefact**	single-file ONNX converted + parity-checked (torch vs ORT max diff 2.0e-06), deployed to `rpi/model/`	■ converted + deployed; runs live on aarch64 ORT — Pi live loop classifies commands with it (Oct 16 log, `evidence/logs_live_20261016.log`)

2. Known limits (stated plainly)

1. **Real-voice out-of-scope is the honest weak point.** The model still mis-accepts real non-command speech (§04); the runtime's `min_conf 0.25` gate fails safe, and the next data round targets real OOS clips.

2. **The CRNN is 0.367 M parameters** — tiny and real-time (62.78 M FLOPs, **9.3 ms mean / 9.4 ms p95 measured on the Pi 4**, prof's 100+100 protocol); the BiGRU was chosen deliberately for temporal modelling.

3. **Wake model is trained mostly on TTS data** (the wake training adds 122 real clips + 694 real-room negative windows; live scores 0.994–0.999, room-neg max 0.065 vs threshold 0.99). Live-mic spoken wake tests on the Pi are still recommended before the demo; the deployed `wake_threshold` is 0.99 (raised from 0.5 after live-room tuning, final value set Oct 16).

4. **Spoken answers play an MP3 (~0.2 s)** — synthesis is offline (`rpi_inf/gen_responses.py`); the runtime only plays files. Music, lights and timer commands answer by *acting*, not speaking, so they are unaffected.

5. **mpv** enables pause/volume; without it, the ffplay fallback plays but cannot pause.

6. **Simulated actions are spoken only** — ALARM, TEMPERATURE, CREATE_REMINDER, CALL, MESSAGE have no physical actuator (by design; the brief's UI was removed). Reminders persist only in memory.

3. Future work (in priority order)

Priority	Item	Why
1	~~Verify the CRNN ONNX on the Pi~~ **done Oct 16**	aarch64 ORT latency measured: CRNN p95 9.36 ms, wake p95 4.85 ms (prof's 100+100 protocol, retrained models)

Priority	Item	Why
2	~~Live-mic wake + command threshold tuning in the actual demo room~~ **done Oct 15**	`wake_threshold` raised 0.5 → 0.75 after live-room tuning; 247 live wakes across 11 sessions; raised again to 0.99 on Oct 16 (final deployed value)
3	Record human clips for the weakest intents flagged by the confusion matrix	closes remaining domain gaps
4	Quantise the CRNN to int8	~4× smaller again; current Pi p95 is already 9.4 ms, so this is optional

4. Deliverable checklist

Requirement (from the brief)	Status
Build a dataset to train VCMs (collective)	■ HF `ai231-me2-voice-commands` + supplementals (36,604 clips / 21.0 h), documented in §02
Build and train a VCM (individual)	■ CRNN 0.367 M (the deployed model), §03–04
Design a benchmark for validating VCMs (collective)	■ speaker-disjoint splits, intent + fine + macro-F1 + per-intent recall + OOS false-accept, §02/§04
Validate the performance of your VCM (individual)	■ test/holdout eval + confusion analysis, §04
Build a real-world demo on RPi4/5 (individual)	■ full loop verified on the Pi; CRNN artefact deployed, §07/§09
VCM must be tiny, real-time on RPi4/5	■ 0.367 M params, 62.78 M FLOPs, **9.36 ms p95 measured on Pi 4**, §05
Standalone — no cloud models	■ all inference on-device; network only for the weather *action* (keyless Open-Meteo), §01
No LLM, no ASR — pure VCM doing commands 1–10	■ 37-label classifier + dispatcher + spoken responses, §01/§07

End of report. Back to the README.